9 slab cache 内存分配全链路实现 2
9.1 slab cache 内存分配流程 2
9.2 slab cache 的快速分配路径 6
9.3 slab cache 的慢速分配路径 8
9.3.1 从本地 cpu 缓存 partial 列表中分配 19
9.3.2 从 NUMA 节点缓存中分配 21
9.3.4 从 NUMA 节点缓存 partial 链表中查找 25
9.3.5 从伙伴系统中重新申请 slab 29
9.4 初始化 slab freelist 链表 35
9.5 初始化 slab 对象 39

9 slab cache 内存分配全链路实现

9.1 slab cache 内存分配流程

使用 fork() 系统调用创建进程的时候，内核需要为进程创建 task_struct 结构，struct task_struct 是内核中的核心数据结构，当然也会有专属的 slab cache 来进行管理，task_struct 的专属 slab cache 为 task_struct_cachep。

以内核从 task_struct_cachep 中申请 task_struct 对象为例，分析 slab cache 分配内存的整个源码实现。内核通过定义在文件 /kernel/fork.c 中的 dup_task_struct 函数来为进程申请 task_struct 结构并初始化。

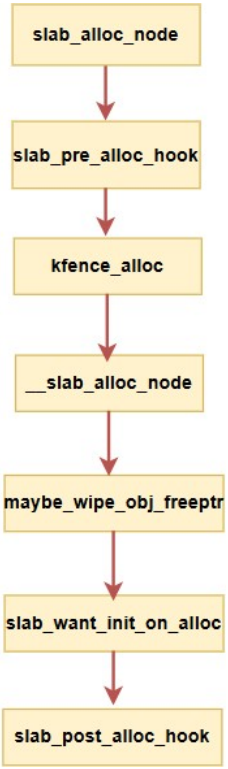
```
static struct task_struct *dup_task_struct(struct task_struct *orig, int
node)
{
    struct task_struct *tsk;
            ......
    // 从 task_struct 对象专属的 slab cache 中申请 task_struct 对象
    tsk = alloc_task_struct_node(node);
            ......
}
```

```
static inline struct task_struct *alloc_task_struct_node(int node)
{
    // 利用 task_struct_cachep 动态分配 task_struct 对象
    return kmem_cache_alloc_node(task_struct_cachep, GFP_KERNEL, node);
}
```

内核中通过 kmem_cache_alloc_node 函数要求 slab cache 从指定的 NUMA 节点中分配对象。分配内存的核心逻辑封装在函数 slab_alloc_node 中。

```
void *kmem_cache_alloc_node(struct kmem_cache *s, gfp_t gfpflags, int node)
{
    void *ret = slab_alloc_node(s, NULL, gfpflags, node, _RET_IP_,
s->object_size);

    trace_kmem_cache_alloc(_RET_IP_, ret, s, gfpflags, node);
    return ret;
}
EXPORT_SYMBOL(kmem_cache_alloc_node);
```

其中，函数 slab_alloc_node 主要如下：




```c
static __fastpath_inline void *slab_alloc_node(struct kmem_cache *s, struct
list_lru *lru,
        gfp_t gfpflags, int node, unsigned long addr, size_t orig_size)
{
    void *object;
    struct obj_cgroup *objcg = NULL;
    bool init = false;
    /*分配前预处理
    过滤分配标志（gfpflags & gfp_allowed_mask），确保符合系统状态。
    检查失败注入（should_failslab），模拟分配失败（调试场景）。
    处理 memcg 管控（memcg_slab_pre_alloc_hook），关联对象到控制组。
    */
    s = slab_pre_alloc_hook(s, lru, &objcg, 1, gfpflags);
    if (unlikely(!s))
        return NULL;
    /*
    KFENCE 调试分配（优先快速路径）。
    若启用 KFENCE（内核内存错误检测工具），优先通过 KFENCE 分配对象，
    用于捕获越界访问等错误。
    */
    object = kfence_alloc(s, orig_size, gfpflags);
    if (unlikely(object))
        goto out;
    /*
    常规分配方式。
```

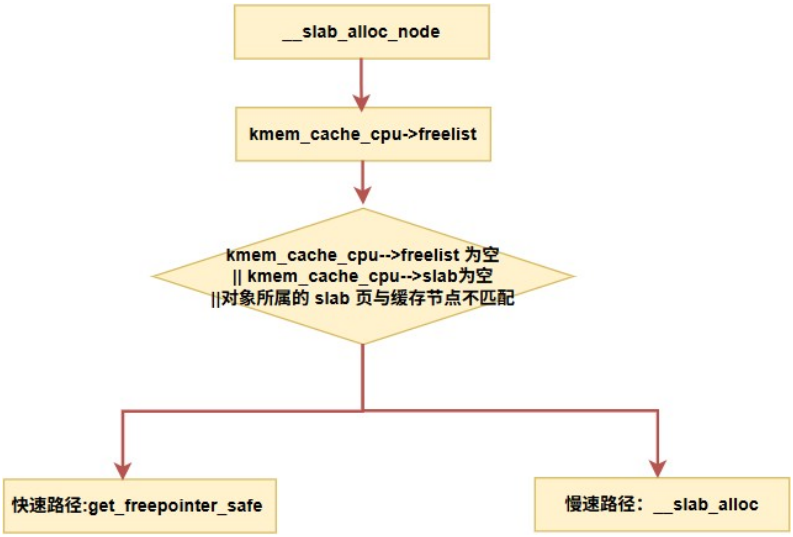
```
    快速路径：尝试从无锁 freelist 链表（kmem_cache_cpu->freelist）直接获取对
象（无锁操作，性能极高）。
    慢速路径：若快速路径失败（如 freelist 链表为空），
    则执行加锁、创建新 slab 页、从其他节点迁移对象等复杂操作。
    */
    object = __slab_alloc_node(s, gfpflags, node, addr, orig_size);

    // 清除对象中的空闲指
    maybe_wipe_obj_freeptr(s, object);

    //判断是否需要初始化（如 kzalloc 需清零）
    init = slab_want_init_on_alloc(gfpflags, s);
out:
    /*
     * 完成分配收尾工作：
     若 init 为 true（如 __GFP_ZERO 标志），
      仅清零 orig_size 字节（而非整个对象，优化性能）。
    记录分配信息（如 KASAN 标记内存为已分配，SLAB_STORE_USER 记录调用栈）。
    关联对象与 memcg（objcg），更新资源统计。
     */
    slab_post_alloc_hook(s, objcg, gfpflags, 1, &object, init, orig_size);
    return object;
}
```

其中，slab cache 的内存分配方式有两个路径：快速分配路径和慢速分配路径。

快速路径：尝试从无锁空闲链表（kmem_cache_cpu->freelist->freelist）直接获取对象。

慢速路径：若快速路径失败（如空闲链表为空），则执行加锁、创建新 slab 页、从其他节点迁移对象等复杂操作。

路径选择的具体源码实现封装在函数__slab_alloc_node 中：



```
static __always_inline void *__slab_alloc_node(struct kmem_cache *s,
        gfp_t gfpflags, int node, unsigned long addr, size_t orig_size)
{
    // slab cache 在当前 cpu 下的本地 cpu 缓存
    struct kmem_cache_cpu *c;
    // object 所在的内存页
    struct slab *slab;
     // 当前 cpu 编号
    unsigned long tid;
    // 用于指向分配成功的对象
    void *object;

redo:
    //无锁分配失败时，跳转至此重试。
    // 获取当前 CPU 专属的 kmem_cache_cpu 实例，无锁访问（CPU 本地数据）。
      // slab cache 首先尝试从当前 cpu 本地缓存 kmem_cache_cpu
    //中获取空闲对象
    c = raw_cpu_ptr(s->cpu_slab);
    // 原子读取当前 CPU 缓存的transaction ID
    tid = READ_ONCE(c->tid);
    //禁止编译器重排序，确保 tid 读取在 c->freelist/c->slab 读取之前。
    //避免因指令重排导致“使用旧 tid 匹配新 freelist” 的错误，
    //确保transaction ID 与空闲链表的关联性。
    //避免因指令重排导致“使用旧 tid 匹配新 freelist” 的错误，
    //确保transaction ID 与空闲链表的关联性。
    barrier();
    // 无锁访问：从 slab cache 的 cpu 本地缓存 kmem_cache_cpu
    //中获取缓存的 slub 空闲对象列表，当前 CPU 对其访问无需加锁。
    // 这里的 freelist 指向本地 cpu 缓存的 slub 中第一个空闲对象
    object = c->freelist;
    // 获取本地 cpu 缓存的 slub，这里用 page 表示，如果是复合页
    //，这里指向复合页的首页 head page
    slab = c->slab;
    if (!USE_LOCKLESS_FAST_PATH() ||
        unlikely(!object || !slab || !node_match(slab, node))) {
        //走慢速路径
        /*
        慢速路径触发条件：
        如果 slab cache 的 cpu 本地缓存中已经没有空闲对象了
         或者 cpu 本地缓存中的 slub 并不属于指定的 NUMA 节点，
        走慢速路径加锁复杂分配。
        处理快速路径无法覆盖的场景，包括：
        // 1. 检查 kmem_cache_cpu 的 partial 列表中是否有空闲的 slub
         // 2. 检查 kmem_cache_node 的 partial 列表中是否有空闲的 slub
```

```
        // 3. 如果都没有，则只能重新到伙伴系统中去申请内存页
        */
        object = __slab_alloc(s, gfpflags, node, addr, c, orig_size);
    } else {
        //快速路径：无锁原子更新
        /*
        // 走到该分支表示，slab cache 的 cpu 本地缓存中还有空闲对象，
        // 直接分配
        // 快速路径 fast path 下分配成功，从当前空闲对象中获取
        // 下一个空闲对象指针 next_object
        */
        // 获取下一个空闲对象指针
        void *next_object = get_freepointer_safe(s, object);
        /*
         更新 kmem_cache_cpu 结构中的 freelist 指向 next_object
        __update_cpu_freelist_fast 核心逻辑：
       原子执行 “验证 + 更新” 操作（通过 cmpxchg_double）：
       验证当前 CPU 缓存的 tid 和 freelist 未被修改（确保无并发干扰）；
       若验证通过，将 freelist 更新为 next_object，并递增 tid（生成新事务 ID）；
       若验证失败（如其他操作修改了 freelist），返回 false，触发重试。
       */
        if (unlikely(!__update_cpu_freelist_fast(s, object, next_object,
tid))) {
            note_cmpxchg_failure("slab_alloc", s, tid);
            goto redo;// 原子更新失败，重试
        }
         // cpu 预取 next_object 的 freepointer 到 cpu 高速缓存，
         //  加快下一次分配对象的速度
        prefetch_freepointer(s, next_object);
        stat(s, ALLOC_FASTPATH); // 统计快速路径分配次数
    }
    return object;
}
```

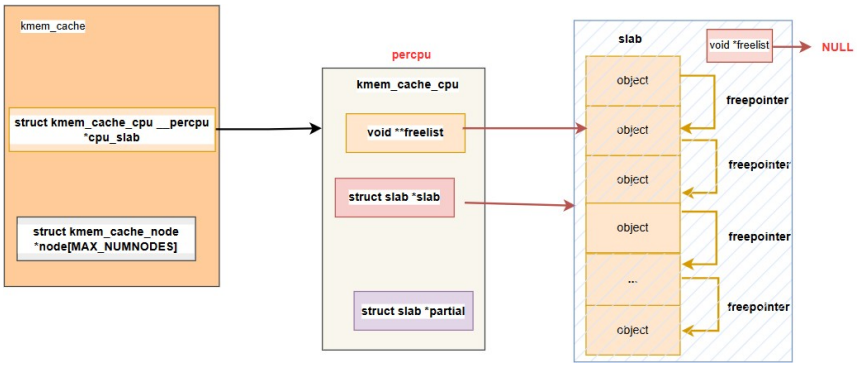
9.2 slab cache 的快速分配路径

slab cache 在最开始会进入 fastpath 分配对象，也就是说首先会从 cpu 本地缓存 kmem_cache_cpu->freelist 中获取对象。在获取 kmem_cache_cpu 结构的时候需要保证这个 cpu 本地缓存是属于当前执行进程的 cpu。

内核通过 kmem_cache_cpu->freelist 来获取 cpu 缓存 slab 中的第一个空闲对象。如果当前 cpu 缓存 slab 是空的（没有空闲对象可供分配）或者该 slab 所在的 NUMA 节点不是指定的。就会通过 __slab_alloc 进入到慢速分配路径 slowpath 中。

如果当前 cpu 缓存 slab 有空闲的对象并且 slab 所在的 NUMA 节点是指定的

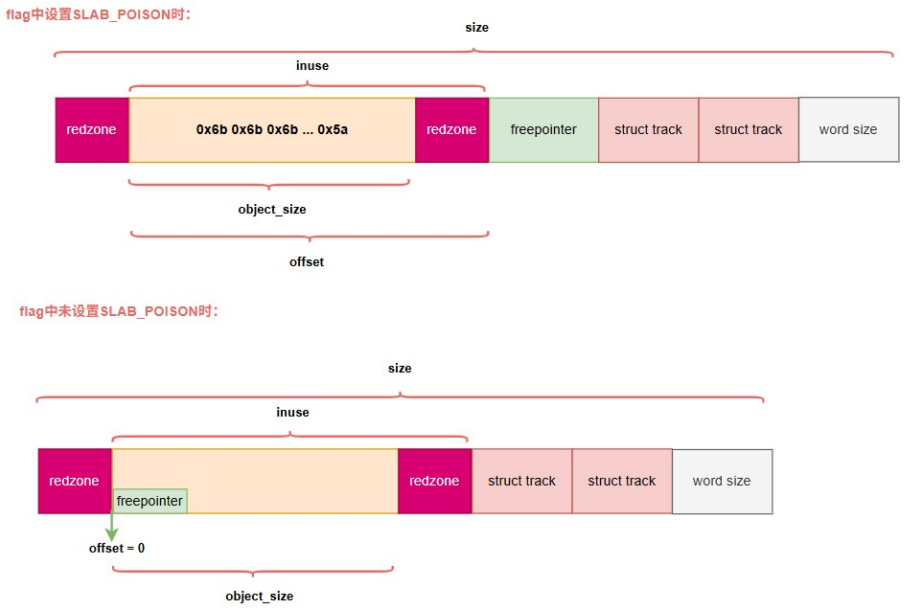
节点，那么将当前 kmem_cache_cpu->freelist 指向的第一个空闲对象从 slab 中拿出，并分配出去。



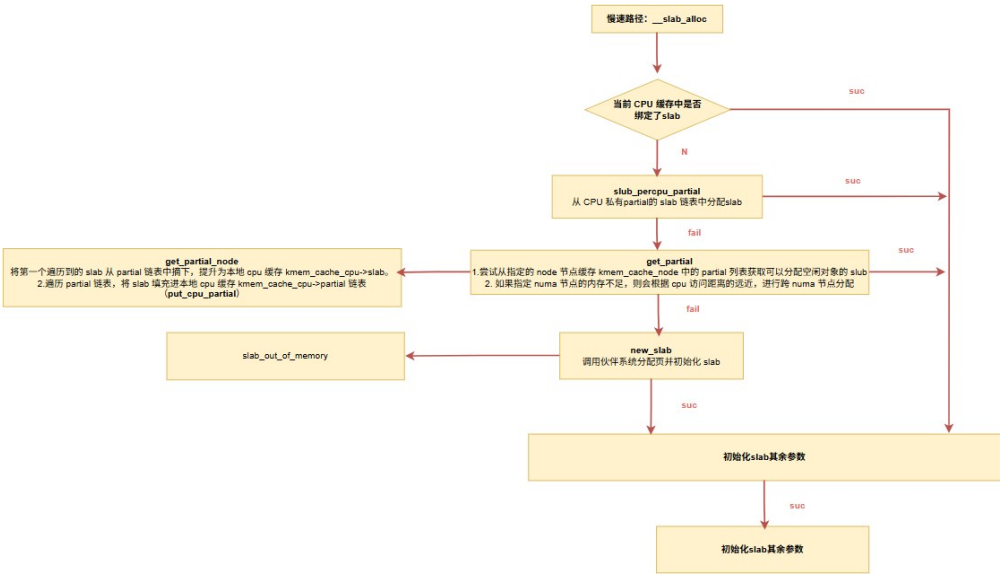
随后通过 get_freepointer_safe 获取当前分配对象的 freepointer 指针（指向其下一个空闲对象），然后将 kmem_cache_cpu->freelist 更新为 freepointer （指向的下一个空闲对象）。

```
// slub 中的空闲对象中均保存了下一个空闲对象的指针 free_pointer
// free_pointor  在 object 中的位置由 kmem_cache 结构的 offset 指定
__no_kmsan_checks
static inline void *get_freepointer_safe(struct kmem_cache *s, void
*object)
{
    // freepointer 在 object 内存区域的起始地址
    unsigned long freepointer_addr;
    // 指向下一个空闲对象的 free_pontier
    freeptr_t p;

    if (!debug_pagealloc_enabled_static())
        return get_freepointer(s, object);
    object = kasan_reset_tag(object);

    // free_pointer 位于 object 起始地址的 offset 偏移处
    freepointer_addr = (unsigned long)object + s->offset;
    copy_from_kernel_nofault(&p, (freeptr_t *)freepointer_addr,
sizeof(p));
     // 返回下一个空闲对象地址
    return freelist_ptr_decode(s, p, freepointer_addr);
}
```

毒化和非毒化下，free_pointer 指针与 object 的相对位置：



9.3 slab cache 的慢速分配路径



慢速路径的入口函数为__slab_alloc。慢速路径主要执行逻辑封装在__slab_alloc 函数中。

```
    static void *__slab_alloc(struct kmem_cache *s, gfp_t gfpflags, int
node,
            unsigned long addr, struct kmem_cache_cpu *c, unsigned int
orig_size)
{
    void *p;
```

```
//CONFIG_PREEMPT_COUNT 启用时，内核支持抢占（高优先级任务可中断当前任务并调度到其他 CPU）。
#ifdef CONFIG_PREEMPT_COUNT
    若调用 __slab_alloc 前未禁用抢占，可能发生以下情况：
    初始传入的 c 是原 CPU 的本地缓存指针；
    任务被抢占后调度到新 CPU，原 c 指向的 CPU 本地缓存已无效（每个 CPU 有独立的 kmem_cache_cpu）。
    解决：通过 slub_get_cpu_ptr(s->cpu_slab) 重新获取当前 CPU 的本地缓存指针，覆盖原 c，确保后续操作使用正确的 CPU 本地数据。
    */
    c = slub_get_cpu_ptr(s->cpu_slab);
#endif

    p = ___slab_alloc(s, gfpflags, node, addr, c, orig_size);
    // 释放 CPU 本地缓存引用（CONFIG_PREEMPT_COUNT 启用时）
    //与 slub_get_cpu_ptr 配对，释放对当前 CPU 本地缓存的引用
    //在支持抢占的内核中，显式释放可避免引用计数泄露，确保 CPU 本地数据的生命周期正确。
#ifdef CONFIG_PREEMPT_COUNT
    slub_put_cpu_ptr(s->cpu_slab);
#endif
    return p;
}
```

___slab_alloc 函数源码实现如下：

```
static void *___slab_alloc(struct kmem_cache *s, gfp_t gfpflags, int node,
              unsigned long addr, struct kmem_cache_cpu *c, unsigned int orig_size)
{
    // 指向 slub 中可供分配的第一个空闲对象
    void *freelist;

     // 空闲对象所在的 slub
    struct slab *slab;
    unsigned long flags;
    struct partial_context pc;

    stat(s, ALLOC_SLOWPATH);
reread_slab:
    从 slab cache 的本地 cpu 缓存中获取缓存的 slub 页
    slab = READ_ONCE(c->slab);
    // 无关联 slab，直接新建
    if (!slab) {
```

```
        //node != NUMA_NO_NODE：检查当前是否指定了一个有效的、具体的节点。
        //node_isset：表示 node 是否是一个有效的 NUMA 节点 ID（如 0、1、2 等，
        //对应系统中的物理节点），操作需要限制在该节点上进行
        //（如内存分配需从 node 节点获取）。
        if (unlikely(node != NUMA_NO_NODE &&
                 !node_isset(node, slab_nodes)))
            node = NUMA_NO_NODE;
       // 跳转到 new_slab 分支进行降级处理走慢速分配路径
        goto new_slab;
    }
    // 1. 节点匹配校验：slab 所属节点是否与指定的目标节点一致
    if (unlikely(!node_match(slab, node))) {
        // 与指定的目标节点不一致，取消节点限制
        if (!node_isset(node, slab_nodes)) {
            node = NUMA_NO_NODE;
    //当目标 NUMA 节点（node）不在 slab_nodes 掩码允许的范围内时，
    //将 node 重置为 NUMA_NO_NODE（表示取消节点限制，允许全局分配）
    //此时并不弃用当前 slab
    /*
      假设系统有 2 个 NUMA 节点（0 和 1），slab_nodes
      掩码为 0b11（允许节点 0 和 1）：
      1）若调用者指定 node=0：node_isset(0, slab_nodes) 为 1，
      不进入分支，继续在节点 0 分配；
      2）若调用者指定 node=2（无效节点）：node_isset(2, slab_nodes) 为 0，
      进入分支，node 被重置为 NUMA_NO_NODE，分配器可在节点 0 或 1 中选择。
      3）若节点 1 因热移除被从 slab_nodes 中清除（掩码变为 0b01），
      此时调用者指定 node=1：node_isset(1, slab_nodes) 为 0，
      进入分支，node 重置为 NUMA_NO_NODE，分配器自动选择节点 0 分配。
   */
        } else {
            stat(s, ALLOC_NODE_MISMATCH);
      //此时目标节点 node 有效，但当前 slab 不属于指定的目标节点
      //此时，拒绝使用其他节点的 slab 页，通过停用不匹配的 slab
      //并重新获取资源，严格保证 NUMA 本地性
            goto deactivate_slab;
        }
    }
    // 2. 内存策略匹配：slab 是否符合分配标志的内存策略
     （如 PFMEMALLOC 紧急内存）
    if (unlikely(!pfmemalloc_match(slab, gfpflags)))
    // 策略不匹配，停用当前 slab
        goto deactivate_slab;
    // 禁用中断并加 CPU 本地锁（确保并发安全）
    local_lock_irqsave(&s->cpu_slab->lock, flags);
```

```
    // 加锁后再次校验 slab 有效性（防止并发修改）
    if (unlikely(slab != c->slab)) {
        local_unlock_irqrestore(&s->cpu_slab->lock, flags);
        goto reread_slab;// slab 已变化，重新读取
    }
    // 从 当前 CPU 本地缓存的空闲列表中读取第一个空闲对象
    freelist = c->freelist;
    // 有空闲对象，直接加载
    if (freelist)
            goto load_freelist;
    //检查 slab->freelist ，
    //因为此时有可能其他 cpu 又释放了一些对象到 slub 中这时
    //slub 对应的  slab->freelist 又不为空，可以直接分配
    //从 cpu 关联的 slab 中获取空闲对象
    freelist = get_freelist(s, slab);

    if (!freelist) {
        //slab 无空闲对象
        // 解除 CPU 与该 slab 的关联
        c->slab = NULL;
        // 更新 transaction ID
        c->tid = next_tid(c->tid);
        local_unlock_irqrestore(&s->cpu_slab->lock, flags);
        stat(s, DEACTIVATE_BYPASS);
        // 新建 slab 页
        goto new_slab;
    }
    stat(s, ALLOC_REFILL);
load_freelist:
    lockdep_assert_held(this_cpu_ptr(&s->cpu_slab->lock));
    // 校验 slab 状态，确保 CPU 独占使用.调试断言宏.
    //断言当前 CPU 本地缓存关联的 slab 必须处于 frozen（冻结）状态，
    //只允许本地 cpu 从中分配对象
    //若不满足则触发内核错误
    VM_BUG_ON(!c->slab->frozen);
    //由于第一个空闲对象马上要被分配出去。
    //更新 c->freelist 指 slab->freelist 中的下一个空闲对象
    c->freelist = get_freepointer(s, freelist);
    // 更新 slab cache 的 cpu 本地缓存分配对象的全局 transaction id
    // 每当分配完一次对象，kmem_cache_cpu 中的 tid 都需要改变
    c->tid = next_tid(c->tid);
    local_unlock_irqrestore(&s->cpu_slab->lock, flags);
    // 返回第一个空闲对象
```

```
    return freelist;
    //停用无效 slab。当 slab 节点 / 策略不匹配或无空闲对象时，
    //停用该 slab 并归还给全局链表
deactivate_slab:
    local_lock_irqsave(&s->cpu_slab->lock, flags);
    // 再次校验 slab 有效性
    if (slab != c->slab) {
        local_unlock_irqrestore(&s->cpu_slab->lock, flags);
        goto reread_slab;
    }
    freelist = c->freelist;
    c->slab = NULL;
    c->freelist = NULL;
    c->tid = next_tid(c->tid);
    local_unlock_irqrestore(&s->cpu_slab->lock, flags);
    deactivate_slab(s, slab, freelist); //将 slab 归还给全局 partial 链表
new_slab:
//获取新 slab 页。若当前 CPU 无有效 slab，按优先级获取新 slab
#ifdef CONFIG_SLUB_CPU_PARTIAL
//启用 CONFIG_SLUB_CPU_PARTIAL 配置时，
//从 CPU 私有 partial 的 slab 链表（per-CPU partial list）获取可用 slab 页
    while (slub_percpu_partial(c)) {// CPU 私有 partial 链表非空
        local_lock_irqsave(&s->cpu_slab->lock, flags);
        if (unlikely(c->slab)) { //当前 CPU 绑定的 slab 非空
            local_unlock_irqrestore(&s->cpu_slab->lock, flags);
            goto reread_slab; // 跳回重新读取 slab 状态
        }
        if (unlikely(!slub_percpu_partial(c))) {//再次检查 cpu 本地 partial
链表是否为空
            local_unlock_irqrestore(&s->cpu_slab->lock, flags);
        // cpu 本地 partial 链表为空，尝试从全局 partial 链表获取
             goto new_objects;
            }
        // 取出 CPU 私有 partial 链表的首个 slab
        slab = slub_percpu_partial(c);
        // 更新 partial 链表头为下一个 slab（移除当前 slab）
        slub_set_percpu_partial(c, slab);
        local_unlock_irqrestore(&s->cpu_slab->lock, flags);
        stat(s, CPU_PARTIAL_ALLOC);// 统计从 CPU partial 链表分配的次数
        // 检查 slab 所属节点是否与目标节点匹配，
        //以及是否符合分配的内存策略（如 PFMEMALLOC）
        if (unlikely(!node_match(slab, node) ||
                !pfmemalloc_match(slab, gfpflags))) {
            slab->next = NULL;//重置 slab 的链表指针
```

```
            // 将不匹配的 slab 归还给全局 partial 链表
            __put_partials(s, slab);
            continue;
        }
        // 冻结 slab，slab->frozen = 1，
      //（标记为 CPU 独占使用，当前 CPU 可无锁操作其空闲链表，禁止其他 CPU 访问）
        freelist = freeze_slab(s, slab);
        // 跳转到加载空闲链表的逻辑，完成分配
        goto retry_load_slab;
         }
#endif
/*
new_objects 是 SLUB 慢速路径的“资源补充入口”，触发时机为：
CPU 本地缓存无关联 slab；
CPU 私有 partial 链表为空（CONFIG_SLUB_CPU_PARTIAL 启用时）。
此时需从更“全局”的资源池获取 slab 页，优先级为：全局 partial 链表 → 新建 slab
页（调用页分配器）。
*/
new_objects:
    pc.flags = gfpflags;// 初始化 partial 上下文：分配标志
    pc.orig_size = orig_size;// 初始化 partial 上下文：原始请求大小
    // 尝试从指定的 node 节点缓存 kmem_cache_node 中的 partial 列表获取可以
分配空闲对象的 slub
    // 如果指定 numa 节点的内存不足，则会根据 cpu 访问距离的远近，进行跨 numa
节点分配
    slab = get_partial(s, node, &pc);
    if (slab) {
        //调试场景特殊处理：
        //如 SLAB_STORE_USER（记录分配 / 释放者）、
        //SLAB_POISON（内存填充）等，需单独分配并标记对象，
        //不直接复用 slab 空闲链表。
        if (kmem_cache_debug(s)) {
            freelist = pc.object;
            if (s->flags & SLAB_STORE_USER)
                set_track(s, freelist, TRACK_ALLOC, addr);
            return freelist;
        }
        // 冻结 slab（标记为当前 CPU 独占）
        freelist = freeze_slab(s, slab);
        goto retry_load_slab;
    }
    //新建 slab 页
    slub_put_cpu_ptr(s->cpu_slab); // 释放当前 CPU 本地缓存引用
    slab = new_slab(s, gfpflags, node);// 调用伙伴系统分配页并初始化 slab
```

```
    c = slub_get_cpu_ptr(s->cpu_slab);// 重新获取 CPU 本地缓存指针
   // 新建失败
    if (unlikely(!slab)) {
        slab_out_of_memory(s, gfpflags, node);
        return NULL;
    }
    stat(s, ALLOC_SLAB); // 统计新建 slab 的次数
    if (kmem_cache_debug(s)) {
        freelist = alloc_single_from_new_slab(s, slab, orig_size);
        if (unlikely(!freelist))
            goto new_objects;
        if (s->flags & SLAB_STORE_USER)
            set_track(s, freelist, TRACK_ALLOC, addr);
        return freelist;
    }
    //新 slab 暂无其他引用，可直接修改元数据（无需原子操作）
    freelist = slab->freelist;// 获取新 slab 的空闲链表头
    slab->freelist = NULL; // 清空 slab 自身的空闲链表（交由 CPU 本地缓存管理）
    slab->inuse = slab->objects;// 标记所有对象为“已使用”（实际会通过分配逐步置为空闲）
    slab->frozen = 1; // 冻结 slab（当前 CPU 独占）
    inc_slabs_node(s, slab_nid(slab), slab->objects);
    if (unlikely(!pfmemalloc_match(slab, gfpflags))) {
    // 若 slab 与分配策略不匹配（如紧急内存策略不匹配）
    // 不加载空闲链表，避免后续分配继续使用不匹配的 slab
    // 停用 slab 并归还给全局链表
        deactivate_slab(s, slab, get_freepointer(s, freelist));
        return freelist;// 仍返回当前对象（本次分配已完成）
    }
    //速路径中重试加载 slab 页到 CPU 本地缓存
    //retry_load_slab 触发时机：当从 CPU 私有 partial 链表、
    //全局 partial 链表获取到 slab，
    //或新建 slab 后，需要将其绑定到当前 CPU 本地缓存（c->slab）时，
    //可能因并发操作（如其他分配 / 释放操作已为 CPU 绑定了 slab）导致冲突，
    //此时通过该标签逻辑解决冲突并重试。
retry_load_slab:
    //加锁保护：禁用中断 + 锁 CPU 本地缓存，确保操作原子性（避免并发修改）
    // 禁用中断并加 CPU 本地锁.确保对 CPU 本地缓存
    //（c->slab、c->freelist 等）的修改是原子操作
    local_lock_irqsave(&s->cpu_slab->lock, flags);
    //校验：当前 CPU 本地缓存是否已关联其他 slab（并发冲突检测）
    if (unlikely(c->slab)) {
        void *flush_freelist = c->freelist;
```

```
        struct slab *flush_slab = c->slab;
        // 解绑原有 slab：清空本地缓存的 slab 和 freelist，更新事务 ID（tid）
        c->slab = NULL;
        c->freelist = NULL;
        c->tid = next_tid(c->tid);
        local_unlock_irqrestore(&s->cpu_slab->lock, flags);
        // 将原 slab 归还给全局链表
        deactivate_slab(s, flush_slab, flush_freelist);
        stat(s, CPUSLAB_FLUSH);
        goto retry_load_slab;// 重试绑定新 slab
    }
    c->slab = slab;// 将新获取的 slab 绑定到 CPU 本地缓存
    goto load_freelist;
}
```

在 slab cache 进入慢速路径之前，内核还需要再次检查本地 cpu 缓存的 slab 的存储容量，确保其真的没有空闲对象了。如果本地 cpu 缓存的 slab 为空（ kmem_cache_cpu->page == null ），直接跳转到 new_slab 分支进入 slow path。

如果本地 cpu 缓存的 slab 不为空，那么需要检查 slab 所属节点是否与目标分配节点一致。如果目标节点不一致，将取消分配节点的限制。如果目标节点一致，则放弃这个 slab，从新从目标节点里的 slab 分配内存。

然后，slab 是否符合分配标志的内存策略，如果不符合，也会弃用这个 slab。如果符合会加锁后再次校验 slab 有效性，若 slab 无效，重新读取 slab；若有效，再次检查 slab 中是否有空闲对象，这么做的目的是因为当前进程可能被中断，当重新调度之后，其他进程可能已经释放了一些对象到缓存 slab 中了，所以在进入 slowpath 之前还是有必要再次检查一下 kmem_cache_cpu->freelist。

如果碰巧，其他进程在当前进程被中断之后，已经释放了一些对象回缓存 slab 中了，那么就直接跳转至 load_freelist 分支，走 fastpath 路径，直接从缓存 slab (kmem_cache_cpu->freelist) 中分配对象，避免进入 slowpath。

```
load_freelist:
    lockdep_assert_held(this_cpu_ptr(&s->cpu_slab->lock));
    // 校验 slab 状态，确保 CPU 独占使用.调试断言宏.
    //断言当前 CPU 本地缓存关联的 slab 必须处于 frozen（冻结）状态，
    //只允许本地 cpu 从中分配对象
    //若不满足则触发内核错误
    VM_BUG_ON(!c->slab->frozen);
    //由于第一个空闲对象马上要被分配出去。
    //更新 c->freelist 指 slab->freelist 中的下一个空闲对象
    c->freelist = get_freepointer(s, freelist);
    // 更新 slab cache 的 cpu 本地缓存分配对象的全局 transaction id
    // 每当分配完一次对象，kmem_cache_cpu 中的 tid 都需要改变
    c->tid = next_tid(c->tid);
```

```
    local_unlock_irqrestore(&s->cpu_slab->lock, flags);
    // 返回第一个空闲对象
    return freelist;
```

如果 kmem_cache_cpu->freelist 还是为空，则需要再次检查 slab 本身的 freelist 是否空，注意这里指的是 struct slab 结构中的 freelist。

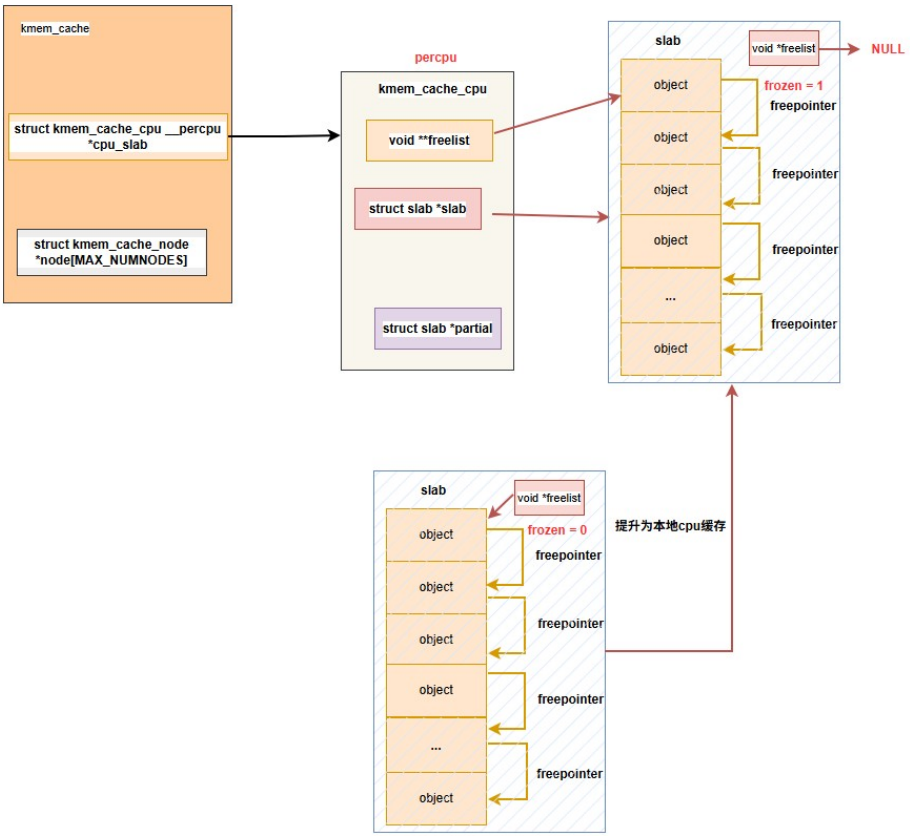
```
struct slab {
     ......
     // 指向内存页中第一个空闲对象
     void *freelist;
     ......
     // 该 slab 是否在对应 slab cache 的本地 CPU 缓存中
    // frozen = 1 表示缓存再本地 cpu 缓存中
     unsigned frozen:1;
    ......
}
```

kmem_cache_cpu 结构中有一个 freelist，slab 结构也有一个 freelist，它们之间的关系是怎样的？

在 slab cache 的整个架构体系中的确存在两个 freelist：

1. 一个是 slab->freelist， slab->freelist 只是单纯的站在 slab 的视角来表示 slab 中的空闲对象列表，这里不考虑 slab 在 slab cache 架构中的位置。

2. 另一个是 kmem_cache_cpu->freelist，特指 slab 被 slab cache 的本地 cpu 缓存之后，slab 中的空闲对象链表。这里可以理解为 slab 中被 cpu 缓存的空闲对象。当 slab 被提升为 cpu 缓存之后，slab->freeelist 直接赋值给 kmem_cache_cpu->freelist，然后 slab->freeelist 置空。slab->frozen 设置为 1，表示 slab 被冻结在当前 cpu 的本地缓存中。



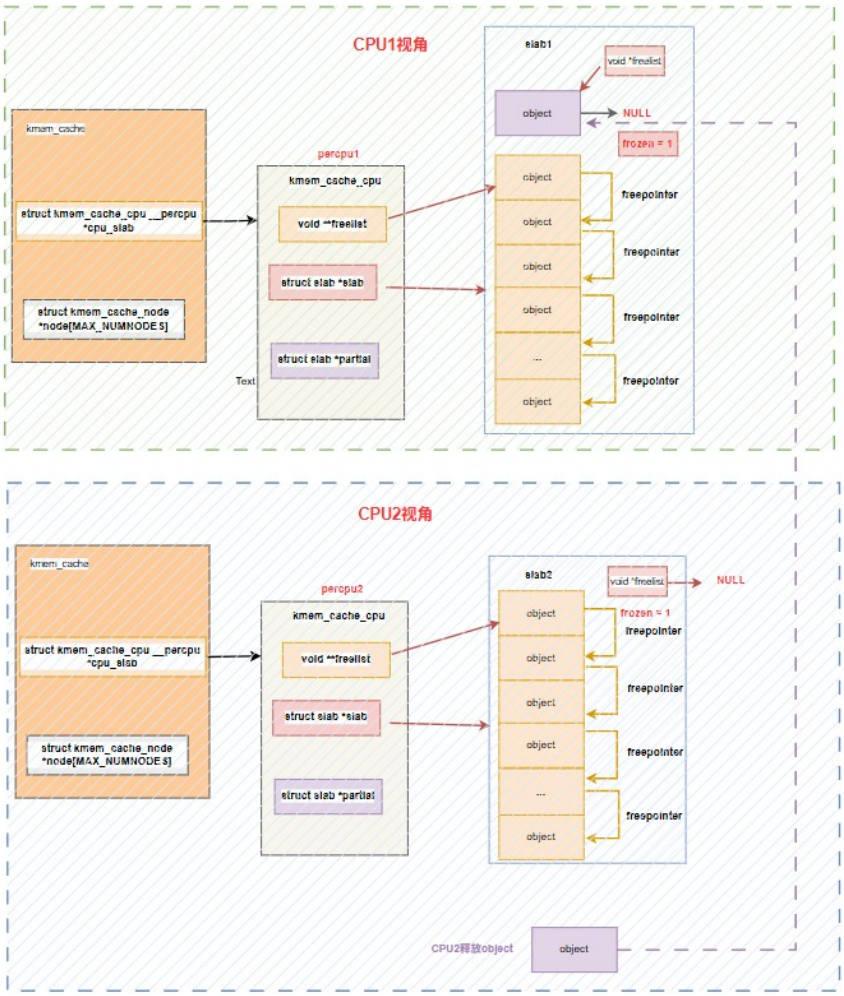
slab 一旦被当前 cpu 缓存，它的状态就变为了冻结状态（slab->frozen = 1），处于冻结状态下的 slab，当前 cpu 可以从该 slab 中分配或者释放对象，但是其他 cpu 只能释放对象到该 slab 中，不能从该 slab 中分配对象。

如果一个 slab 被一个 cpu 缓存之后，那么这个 cpu 在该 slab 看来就是本地 cpu，当本地 cpu 释放对象回这个 slab 的时候会释放回 kmem_cache_cpu->freelist 链表中

如果其他 cpu 想要释放对象回该 slab 时，其他 cpu 只能将对象释放回该 slab 的 slab->freelist 中。

如下图所示，cpu1 在本地缓存了 slab1，cpu2 在本地缓存了 slab2，进程先从 slab1 中获取了一个对象，正常情况下如果进程一直在 cpu1 上运行的话，当进程释放该对象回 slab1 中时，会直接释放回 kmem_cache_cpu1->freelist 链表中。

但如果进程在 slab1 中获取完对象之后，被调度到了 cpu2 上运行，这时进程想要释放对象回 slab1 中时，就不能走快速路径了，因为 cpu2 本地缓存的是 slab2，所以 cpu2 只能将对象释放至 slab1->freelist 中。



这种情况下，在 slab1 的内部视角里，就有了两个 freelist 链表，它们的共同之处都是用于组织 slab1 中的空闲对象，但是 kmem_cache_cpu1->freelist 链表中组织的是缓存在 cpu1 本地的空闲对象，slab1->freelist 链表组织的是由其他 cpu 释放的空闲对象。

在___slab_alloc 函数的开始处，首先内核会在 slab cache 的本地 cpu 缓存 kmem_cache_cpu->freelist 中查找是否有空闲对象，如果这里没有，内核会继续到 slab->freelist 中查看是否有其他 cpu 释放的空闲对象

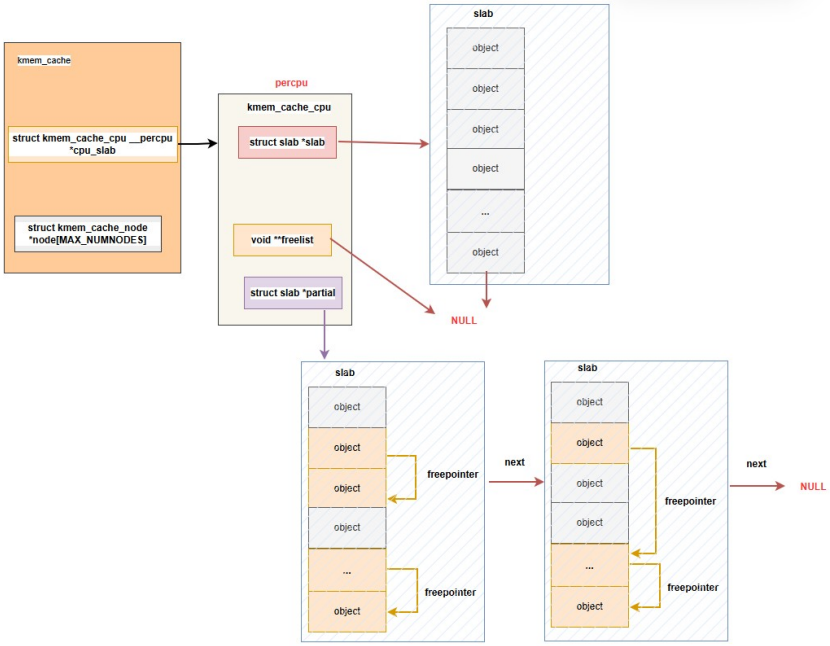
如果两个 freelist 链表都没有空闲对象了，那就证明 slab cache 在当前 cpu 本地缓存中的 slab 已经为空了，将该 slab 从当前 cpu 本地缓存中脱离解冻，程序跳转到 new_slab 分支进入慢速分配路径。

```
static inline void *get_freelist(struct kmem_cache *s, struct slab *slab)
{
    // 用于存放要更新的 slab 属性值
    struct slab new;
    unsigned long counters;
    void *freelist;

    lockdep_assert_held(this_cpu_ptr(&s->cpu_slab->lock));
    do {
```

```
      // 获取 slab 结构的 freelist，当其他 cpu 向 slab 释放
      //对象时 freelist 指向被释放的空闲对象
     //当 slab 被 slab cache 的 cpu 本地缓存时，freelist 置为 null
        freelist = slab->freelist;
        counters = slab->counters;
        new.counters = counters;
        VM_BUG_ON(!new.frozen);
       // 更新 inuse 字段，表示 slab 中的对象 objects 全部被分配出去了
        new.inuse = slab->objects;

    // 如果 freelist != null，表示其他 cpu 又释放了
    //一些对象到 slub 中。
   // 则 slab->frozen = 1 , slub 依然冻结在 cpu 本地缓存中
   // 如果 freelist == null,则 slab->frozen = 0，
    //slub 从 cpu 本地缓存中脱离解冻
        new.frozen = freelist != NULL;
    // 最后原子更新 slab 结构中的相应属性
    // 需要注意的是，当 slab 被 slab cache 本地 cpu 缓存时，
    //slab->freelist 需要置空。
   // 因为在本地 cpu 缓存场景下 slab->freelist
    //指向其他 cpu 释放的空闲对象列表
   // kmem_cache_cpu->freelist 指向的是被本地 cpu 缓存的空闲对象列表
   // 这两个列表中的空闲对象共同组成了 slub 中的空闲对象
    } while (!__slab_update_freelist(s, slab,
        freelist, counters,
        NULL, new.counters,
        "get_freelist"));
    return freelist;
}
```

9.3.1 从本地 cpu 缓存 partial 列表中分配

经过 reread_slab 分支检查，现在已经确认了 slab cache 在当前 cpu 本地缓存的 slab 已经没有任何可供分配的空闲对象了。下面内核正式进入到 slowpath 开始分配对象，首先内核会到本地 cpu 缓存的 partial 列表中去查看是否有一个 slab 可以分配对象。这里内核会从 partial 列表中的头结点开始遍历直到找到一个可以满足分配的 slab 出来。、



随后内核会将该 slab 从 partial 列表中摘下，直接提升为新的本地 cpu 缓存，这样一来 slab cache 的本地 cpu 缓存就被更新了，内核通过 kmem_cache_cpu->freelist 指针将缓存 slab 中的第一个空闲对象分配出去，随后更新 kmem_cache_cpu->freelist 指向 slab 中的下一个空闲对象。

```
new_slab:
//获取新 slab 页。若当前 CPU 无有效 slab，按优先级获取新 slab
#ifdef CONFIG_SLUB_CPU_PARTIAL
//启用 CONFIG_SLUB_CPU_PARTIAL 配置时，
//从 CPU 私有 partial 的 slab 链表（per-CPU partial list）获取可用 slab 页
    while (slub_percpu_partial(c)) {// CPU 私有 partial 链表非空
        local_lock_irqsave(&s->cpu_slab->lock, flags);
        if (unlikely(c->slab)) { //当前 CPU 绑定的 slab 非空
            local_unlock_irqrestore(&s->cpu_slab->lock, flags);
            goto reread_slab; // 跳回重新读取 slab 状态
        }
        if (unlikely(!slub_percpu_partial(c))) {//再次检查 cpu 本地 partial
链表是否为空
            local_unlock_irqrestore(&s->cpu_slab->lock, flags);
        // cpu 本地 partial 链表为空，尝试从全局 partial 链表获取
             goto new_objects;
            }
        // 取出 CPU 私有 partial 链表的首个 slab
        slab = slub_percpu_partial(c);
        // 更新 partial 链表头为下一个 slab（移除当前 slab）
        slub_set_percpu_partial(c, slab);
        local_unlock_irqrestore(&s->cpu_slab->lock, flags);
```

```
        stat(s, CPU_PARTIAL_ALLOC);// 统计从 CPU partial 链表分配的次数
        // 检查 slab 所属节点是否与目标节点匹配，
        //以及是否符合分配的内存策略（如 PFMEMALLOC）
        if (unlikely(!node_match(slab, node) ||
                 !pfmemalloc_match(slab, gfpflags))) {
            slab->next = NULL;//重置 slab 的链表指针
            // 将不匹配的 slab 归还给全局 partial 链表
            __put_partials(s, slab);
            continue;
        }
        // 冻结 slab，slab->frozen = 1，
      //（标记为 CPU 独占使用，当前 CPU 可无锁操作其空闲链表，禁止其他 CPU 访问）
        freelist = freeze_slab(s, slab);
        // 跳转到加载空闲链表的逻辑，完成分配
        goto retry_load_slab;
         }
#endif
```

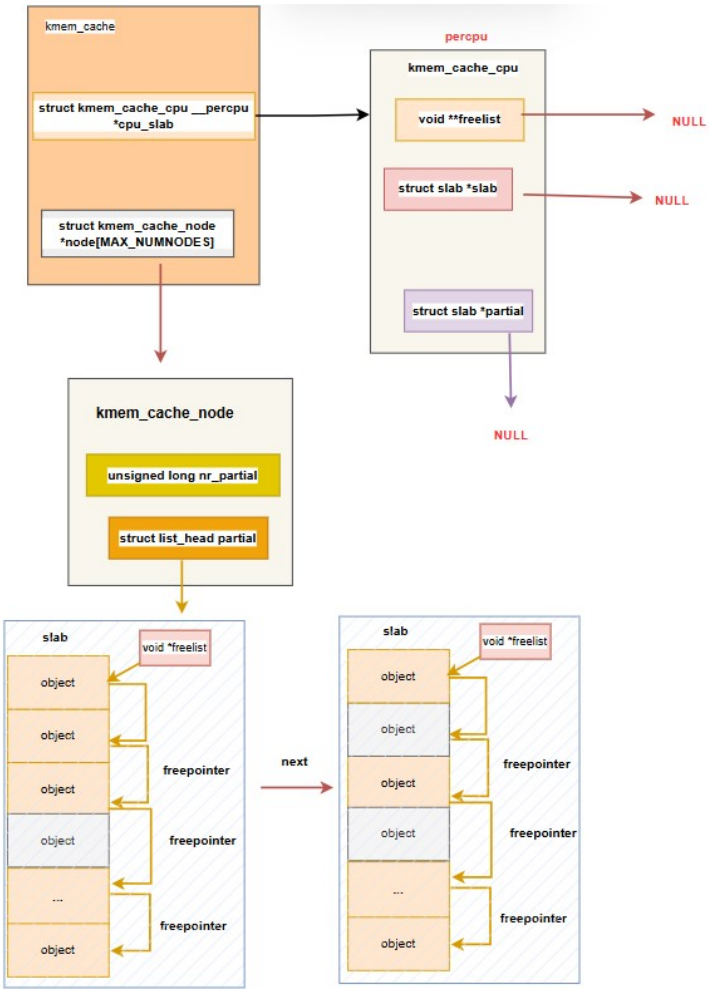
内核对 kmem_cache_cpu->partial 链表的相关操作，定义在文件 mm\slab.h 中：

```
#ifdef CONFIG_SLUB_CPU_PARTIAL

// 获取 slab cache 本地 cpu 缓存的 partial 列表
#define slub_percpu_partial(c)          ((c)->partial)
// 将 partial 列表中第一个 slab 摘下，提升为 cpu 本地缓存，
//用于后续快速分配对象
#define slub_set_percpu_partial(c, p)       \
({                      \
    slub_percpu_partial(c) = (p)->next; \
})
```

如果 slab cache 本地 cpu 缓存 kmem_cache_cpu->partial 链表也是空的，接下来内核就只能到对应 NUMA 节点缓存中去分配对象了。

9.3.2 从 NUMA 节点缓存中分配

当slab cache的本地cpu地缓存的 slab 和本地 cpu 缓存 kmem_cache_cpu->partial 链表都为空时，会从对应 NUMA 节点缓存中分配对象。




```
new_objects:
    pc.flags = gfpflags;// 初始化 partial 上下文：分配标志
    pc.orig_size = orig_size;// 初始化 partial 上下文：原始请求大小
    // 尝试从指定的 node 节点缓存 kmem_cache_node 中的 partial 列表获取可以
分配空闲对象的 slub
    // 如果指定 numa 节点的内存不足，则会根据 cpu 访问距离的远近，
    //进行跨 numa 节点分配
    slab = get_partial(s, node, &pc);
    if (slab) {
        //调试场景特殊处理：
        //如 SLAB_STORE_USER（记录分配 / 释放者）、
        //SLAB_POISON（内存填充）等，需单独分配并标记对象，
        //不直接复用 slab 空闲链表。
        if (kmem_cache_debug(s)) {
            freelist = pc.object;
            if (s->flags & SLAB_STORE_USER)
                set_track(s, freelist, TRACK_ALLOC, addr);
            return freelist;
        }
```

```
        // 冻结 slab（标记为当前 CPU 独占）
        freelist = freeze_slab(s, slab);
        goto retry_load_slab;
    }
    //新建 slab 页
    slub_put_cpu_ptr(s->cpu_slab); // 释放当前 CPU 本地缓存引用
    // 流程走到这里说明 numa cache 里缓存的 slub 也用尽了，
    //无法找到可以分配对象的 slub 了
    // 只能向底层伙伴系统重新申请内存页（slub），然后从新的 slub 中分配对象
    slab = new_slab(s, gfpflags, node);// 调用伙伴系统分配页并初始化 slab
    c = slub_get_cpu_ptr(s->cpu_slab);// 重新获取 CPU 本地缓存指针
   // 新建失败
    if (unlikely(!slab)) {
        slab_out_of_memory(s, gfpflags, node);
        return NULL;
    }
    stat(s, ALLOC_SLAB); // 统计新建 slab 的次数
    if (kmem_cache_debug(s)) {
        freelist = alloc_single_from_new_slab(s, slab, orig_size);
        if (unlikely(!freelist))
            goto new_objects;
        if (s->flags & SLAB_STORE_USER)
            set_track(s, freelist, TRACK_ALLOC, addr);
        return freelist;
    }
    //新 slab 暂无其他引用，可直接修改元数据（无需原子操作）
    freelist = slab->freelist;// 获取新 slab 的空闲链表头
    slab->freelist = NULL; // 清空 slab 自身的空闲链表（交由 CPU 本地缓存管理）
    slab->inuse = slab->objects;// 标记所有对象为“已使用”（实际会通过分配逐步置为空闲）
    slab->frozen = 1; // 冻结 slab（当前 CPU 独占）
    inc_slabs_node(s, slab_nid(slab), slab->objects);
    if (unlikely(!pfmemalloc_match(slab, gfpflags))) {
    // 若 slab 与分配策略不匹配（如紧急内存策略不匹配）
    // 不加载空闲链表，避免后续分配继续使用不匹配的 slab
    // 停用 slab 并归还给全局链表
        deactivate_slab(s, slab, get_freepointer(s, freelist));
        return freelist;// 仍返回当前对象（本次分配已完成）
    }
```

内核首先会在 get_partial 函数中找到我们指定的 NUMA 节点缓存结构 kmem_cache_node ，然后开始遍历 kmem_cache_node->partial 链表直到找到一个可供分配对象的 slab。然后将这个 slab 提升为 slab cache 的本地 cpu 缓存，并从 kmem_cache_node->partial 链表中依次填充 slab 到 kmem_cache_cpu->partial。

如果我们指定的 NUMA 节点 kmem_cache_node->partial 链表也是空的，随后内核就会跨 NUMA 节点进行查找，按照访问距离由近到远，开始查找其他 NUMA 节点 kmem_cache_node->partial 链表。

```
/*
 * Get a slab from somewhere. Search in increasing NUMA distances.
 */
static struct slab *get_any_partial(struct kmem_cache *s,
                   struct partial_context *pc)
{
#ifdef CONFIG_NUMA
    struct zonelist *zonelist;
    struct zoneref *z;
    struct zone *zone;
    enum zone_type highest_zoneidx = gfp_zone(pc->flags);
    struct slab *slab;
    unsigned int cpuset_mems_cookie;
    //用概率控制触发从远程 NUMA 节点获取内存的频率。
    if (!s->remote_node_defrag_ratio ||
            get_cycles() % 1024 > s->remote_node_defrag_ratio)
        return NULL;
    //遍历内存域找远程 partial slab
    do {
        cpuset_mems_cookie = read_mems_allowed_begin();
          //获取指定节点的内存域链表
          //mempolicy_slab_node: 获取当前内存策略指定的 NUMA 节点
        zonelist = node_zonelist(mempolicy_slab_node(), pc->flags);
        //遍历内存域找可用节点
        for_each_zone_zonelist(zone, z, zonelist, highest_zoneidx) {
            struct kmem_cache_node *n;
            //获取缓存池 s 在该节点的 kmem_cache_node
           // zone_to_nid: 获取该内存域所属的
            n = get_node(s, zone_to_nid(zone));
            if (n && cpuset_zone_allowed(zone, pc->flags) &&
                    n->nr_partial > s->min_partial) {
                //从该节点的 partial 链表中取出一个 partial slab
                slab = get_partial_node(s, n, pc);
                if (slab) {

                    return slab;
                }
            }
        }
    } while (read_mems_allowed_retry(cpuset_mems_cookie));
#endif  /* CONFIG_NUMA */
```

```
    return NULL;
}
```

如果还是不行，最后就只能通过 new_slab 函数到伙伴系统中重新申请一个 slab，并将这个 slab 提升为本地 cpu 缓存。

9.3.4 从 NUMA 节点缓存 partial 链表中查找

get_partial 函数的主要内容是选取合适的 NUMA 节点缓存，优先使用指定的 NUMA 节点，如果指定的 NUMA 节点中没有足够的内存，内核就会跨 NUMA 节点按照访问距离的远近，选取一个合适的 NUMA 节点。

```
/*
 * Get a partial slab, lock it and return it.
 */
static struct slab *get_partial(struct kmem_cache *s, int node,
                struct partial_context *pc)
{
   // 从指定 node 的 kmem_cache_node 缓存中的 partial 列表中获取到的对象
    struct slab *slab;
     // 即将要所搜索的 kmem_cache_node 缓存对应的 numa node
    int searchnode = node;
    //若未指定节点，查找当前 CPU / 进程所属的内存节点 ID
    if (node == NUMA_NO_NODE)
        searchnode = numa_mem_id();
    //从指定节点的 partial 链表中取出一个可用的 partial slab
    slab = get_partial_node(s, get_node(s, searchnode), pc);
    //若当前指定的节点 searchnode 无可用 partial slab
    if (slab || node != NUMA_NO_NODE)
        return slab;
   //调用 get_any_partial 尝试从远程节点获取 partial slab
    return get_any_partial(s, pc);
}
```

get_partial 函数的主要内容是选取合适的 NUMA 节点缓存，优先使用指定的 NUMA 节点，如果指定的 NUMA 节点中没有足够的内存，内核就会跨 NUMA 节点按照访问距离的远近，选取一个合适的 NUMA 节点。

然后通过 get_partial_node 在选取的 NUMA 节点缓存 kmem_cache_node->partial 链表中查找 slab，并将 slab 从 NUMA 节点缓存 partial 链表中摘取下来。

```
/*
 * Try to allocate a partial slab from a specific node.
 */
static struct slab *get_partial_node(struct kmem_cache *s,
                    struct kmem_cache_node *n,
                    struct partial_context *pc)
{
```

```
  // 接下来就会挨个遍历 kmem_cache_node 的 partial 列表中的 slub
  // slab，slab2 这两个变量用于临时存储遍历的 slub
  // partial 用于指向从 partial 列表 slub 中申请到的对象
  struct slab *slab, *slab2, *partial = NULL;
  unsigned long flags;
// 用于记录 slab cache 本地 cpu 缓存 kmem_cache_cpu
// 中所缓存的空闲对象总数（包括 partial 列表）
// 后续会向 kmem_cache_cpu 中填充 slub
  unsigned int partial_slabs = 0;
  if (!n || !n->nr_partial)
      return NULL;
  spin_lock_irqsave(&n->list_lock, flags);
  //开始挨个遍历 kmem_cache_node 的 partial 列表，
  //获取 slub 用于分配对象以及填充 kmem_cache_cpu
  list_for_each_entry_safe(slab, slab2, &n->partial, slab_list) {
      if (!pfmemalloc_match(slab, pc->flags))
          continue;
 //SLUB_TINY / 调试模式：直接分配单个对象
      if (IS_ENABLED(CONFIG_SLUB_TINY) || kmem_cache_debug(s)) {
 //slab 表示当前遍历到的 slab，这里会从该 slab 中获取空闲对象赋值给 object
// 并将 slab 从 kmem_cache_node 的 partial 列表上摘下
          void *object = alloc_single_from_partial(s, n, slab,
                                  pc->orig_size);

          if (object) {
    // 分配成功：记录该 slab 为 partial，将对象指针存入 pc->object，
    //跳出遍历（只取一个对象即可）；
              partial = slab;
              pc->object = object;
              break;
          }
     //分配失败：跳过该 slab，继续遍历下一个 slab。
          continue;
      }
      /* 普通模式：取出 slab 并补充到 CPU 局部缓存 */
      // 将 slab 从节点 partial 链表移除
      remove_partial(n, slab);
      if (!partial) {
          // 记录该 slab 为 partial
          partial = slab;
          // 统计：从 partial 分配的次数
          stat(s, ALLOC_FROM_PARTIAL);
      } else {
```

```
            // 将 slab 补充到 CPU 局部 partial 缓存
            put_cpu_partial(s, slab, 0);
            stat(s, CPU_PARTIAL_NODE);
             // 统计补充的 slab 数量
            partial_slabs++;
        }
#ifdef CONFIG_SLUB_CPU_PARTIAL
           //控制补充的 slab 数量。
           //补充的 slab 数量超过 CPU 阈值的一半时，停止遍历
        if (!kmem_cache_has_cpu_partial(s)
            || partial_slabs > s->cpu_partial_slabs / 2)
            break;
#else
        break;
#endif
    }
    spin_unlock_irqrestore(&n->list_lock, flags);
    return partial;
}
```

get_partial_node 函数通过遍历 NUMA 节点缓存结构 kmem_cache_node->partial 链表主要做两件事情：

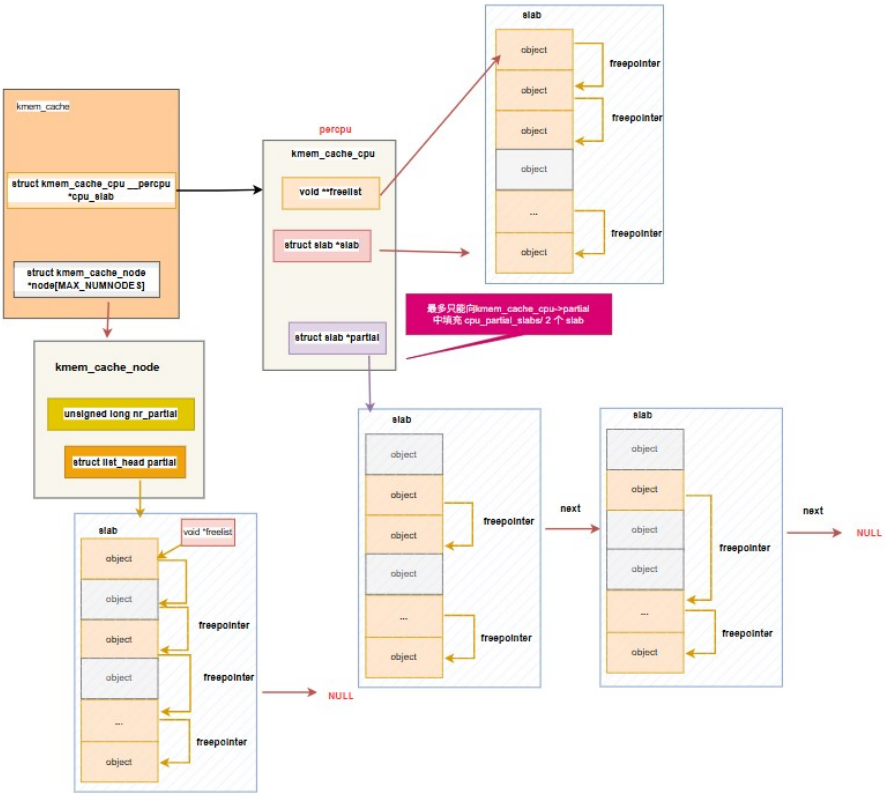
1. 将第一个遍历到的 slab 从 partial 链表中摘下，提升为本地 cpu 缓存 kmem_cache_cpu->slab。（但是，SLUB_TINY / 调试模式下会直接跳出循环，不会执行第二步。）

2. 继续遍历 partial 链表，后面遍历到的 slab 会填充进本地 cpu 缓存 kmem_cache_cpu->partial 链表中，直到当前 cpu 缓存的所有空闲对象数目 partial_slabs（既包括 kmem_cache_cpu->page 中的空闲对象也包括 kmem_cache_cpu->partial 链表中的空闲对象）超过了 **kmem_cache->cpu_partial / 2** 的限制。

SLUB_TINY / 调试模式下 alloc_single_from_partial 调用函数从 NUMA 节点 partial 链表的某个 slab 中分配单个对象。

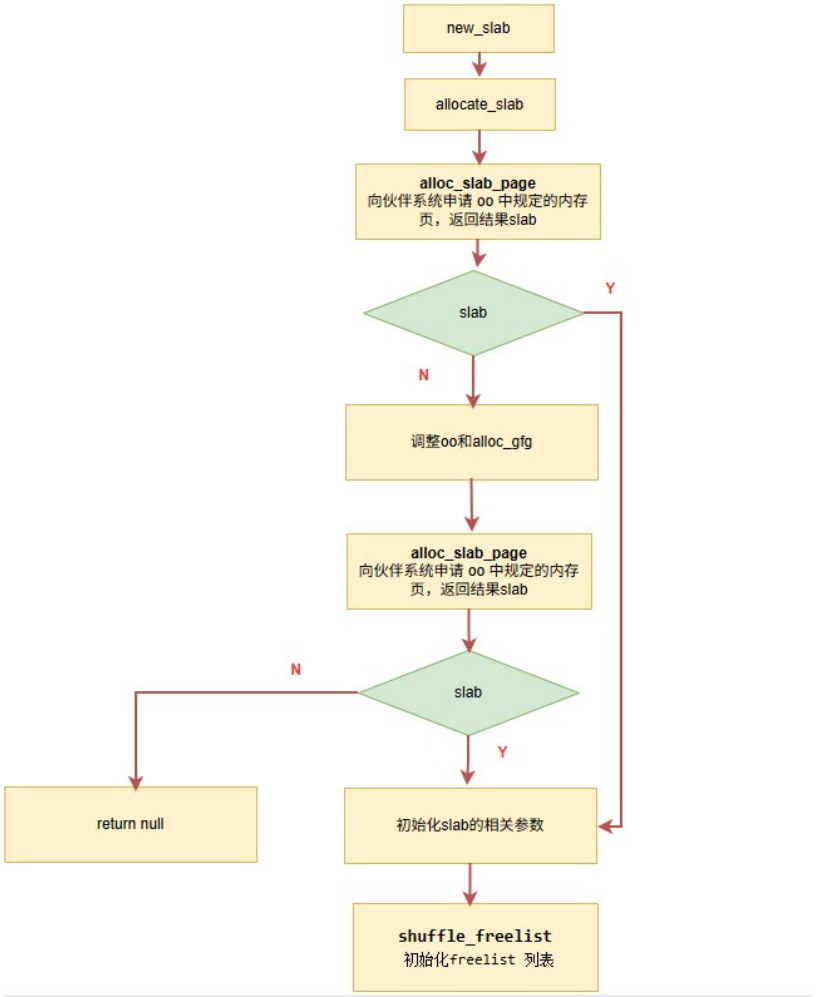
```
    / 从 kmem_cache_node 的 partial 列表中摘下一个 slub 分配对象
// 随后将摘下的 slub 放入 cpu 本地缓存 kmem_cache_cpu 中缓存，
//后续分配对象直接就会 cpu 缓存中分配
static void *alloc_single_from_partial(struct kmem_cache *s,
        struct kmem_cache_node *n, struct slab *slab, int orig_size)
{
    void *object;

    lockdep_assert_held(&n->list_lock);
     // slab 表示即将从 kmem_cache_node 的 partial 列表摘下的 slub
    // 获取 slub  中的空闲对象列表 freelist
    object = slab->freelist;
    //object 马上要被分配出去了，更新 freelist 指针
    slab->freelist = get_freepointer(s, object);
```

```
    //slab 的已用对象数+1
    slab->inuse++;
    if (!alloc_debug_processing(s, slab, object, orig_size)) {
    //调用 remove_partial 将该 slab 从 partial 链表移除
        remove_partial(n, slab);
        return NULL;
    }
    //检查 slab 是否分配满。
    //已用对象数 == slab 总对象数 ，表示该 slab 无空闲对象
    if (slab->inuse == slab->objects) {
        //将 slab 从 partial 链表移除
        remove_partial(n, slab);
        // 加入节点的 full 链表中
        add_full(s, n, slab);
    }
    return object;
}
```

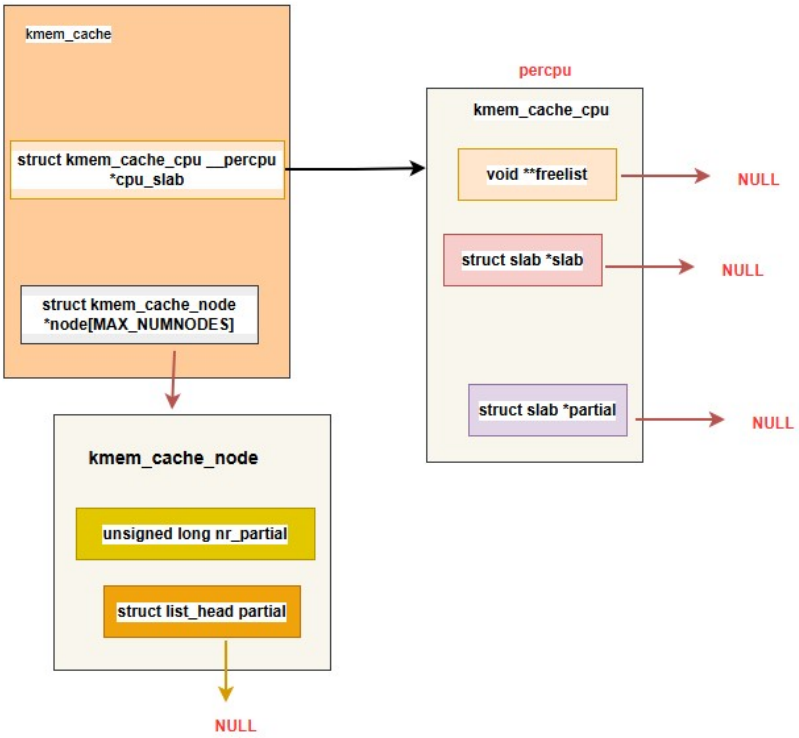


现在 slab cache 的本地 cpu 缓存已经被填充好了，随后内核会从 kmem_cache_cpu->freelist 中分配一个空闲对象给进程使用。

9.3.5 从伙伴系统中重新申请 slab



若本地 cpu 缓存 kmem_cache_cpu->slab 为空，kmem_cache_cpu->partial 为空，kmem_cache_node->partial 链表也为空，比如 slab cache 在刚刚被创建出来的时候就是这个架构。



在这种情况下，内核就需要通过 new_slab 函数到伙伴系统中申请一个新的 slab，填充到 slab cache 的本地 cpu 缓存 kmem_cache_cpu->page 中。

```
    static struct slab *new_slab(struct kmem_cache *s, gfp_t flags, int node)
{
    if (unlikely(flags & GFP_SLAB_BUG_MASK))
        flags = kmalloc_fix_flags(flags);

    WARN_ON_ONCE(s->ctor && (flags & __GFP_ZERO));
    return allocate_slab(s,
        flags & (GFP_RECLAIM_MASK | GFP_CONSTRAINT_MASK), node);
}
```

allocate_slab 函数实现如下，它是 Linux 内核 SLUB 分配器中实际完成新 slab 物理页分配和全量元数据初始化的核心函数：

```
static struct slab *allocate_slab(struct kmem_cache *s, gfp_t flags, int
node)
{
    // 用于指向从伙伴系统中申请到的内存页
    struct slab *slab;
        // kmem_cache 结构的中的 kmem_cache_order_objects oo，
        //表示该 slub 需要多少个内存页，以及能够容纳多少个对象
        // kmem_cache_order_objects 的高 16 位表示需要的内存页个数，
          低 16 位表示能够容纳的对象个数
    struct kmem_cache_order_objects oo = s->oo;
    // 控制向伙伴系统申请内存的行为规范掩码
```

```
    gfp_t alloc_gfp;
    void *start, *p, *next;
    int idx;
    bool shuffle;

    flags &= gfp_allowed_mask;
    flags |= s->allocflags;
    alloc_gfp = (flags | __GFP_NOWARN | __GFP_NORETRY) & ~__GFP_NOFAIL;
    if ((alloc_gfp & __GFP_DIRECT_RECLAIM) && oo_order(oo) >
oo_order(s->min))
        alloc_gfp = (alloc_gfp | __GFP_NOMEMALLOC) & ~__GFP_RECLAIM;

    // 向伙伴系统申请 oo 中规定的内存页
    slab = alloc_slab_page(alloc_gfp, node, oo);
    if (unlikely(!slab)) {
     // 如果伙伴系统无法满足正常情况下 oo 指定的内存页个数
     // 那么这里再次尝试用 min 中指定的内存页个数向伙伴系统申请内存页
     // min 表示当内存不足或者内存碎片的原因无法满足内存分配时，
     //至少要保证容纳一个对象所使用内存页个数
        oo = s->min;
        alloc_gfp = flags;
         // 再次向伙伴系统申请容纳一个对象所需要的内存页（降级）
        slab = alloc_slab_page(alloc_gfp, node, oo);
        if (unlikely(!slab))
            // 如果内存还是不足，直接返回 null
            return NULL;
        stat(s, ORDER_FALLBACK);
    }
    //流程做东这里，表明内存页已经分配成功
    // 初始化 slub 对应的 struct slab 结构中的属性
    // 获取 slub 可以容纳的对象个数
    slab->objects = oo_objects(oo);
    slab->inuse = 0;
    slab->frozen = 0;
    account_slab(slab, oo_order(oo), s, flags);
     // 将 slub cache  与 slab 结构关联
    slab->slab_cache = s;
    //用 0xFC 填充 slub 中的内存，用于内核对内存访问越界检查
    kasan_poison_slab(slab);
     // 获取内存页对应的虚拟内存地址
    start = slab_address(slab);
    setup_slab_debug(s, slab, start);
    // 在配置了 CONFIG_SLAB_FREELIST_RANDOM 选项的情况下
    // 会在 slub 的空闲对象中以随机的顺序初始化 freelist 列表
```

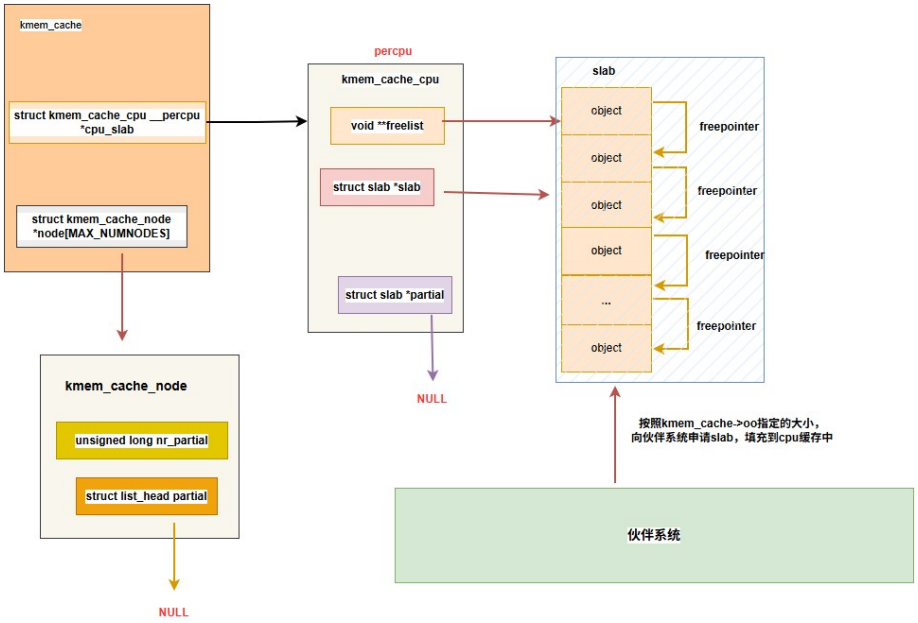
```
    // 返回值 shuffle = true 表示随机初始化 freelist，
    //随机化的执行逻辑已经在 shuffle_freelist 完成后，在返回 true。
    //否则，直接 shuffle = false 表示按照正常的顺序初始化 freelist
    shuffle = shuffle_freelist(s, slab);

    // shuffle = false 则按照正常的顺序来初始化 freelist
    if (!shuffle) {
    // 获取 slub 第一个空闲对象的真正起始地址
    // slub 可能配置了 SLAB_RED_ZONE，
    //这样会在 slub 对象内存空间两侧填充 red zone，防止内存访问越界
    // 这里需要跳过 red zone 获取真正存放对象的内存地址
        start = fixup_red_left(s, start);
         // 填充对象的内存区域以及初始化空闲对象
        start = setup_object(s, start);
     // 用 slub 中的第一个空闲对象作为 freelist 的头结点，
    // 而不是随机的一个空闲对象
        slab->freelist = start;
      // 从 slub 中的第一个空闲对象开始，
      //按照正常的顺序通过对象的 freepointer 串联起 freelist
        for (idx = 0, p = start; idx < slab->objects - 1; idx++) {
            // 获取下一个对象的内存地址
            next = p + s->size;
            // 通过 p 的 freepointer 指针指向 next，
            //设置 p 的下一个空闲对象为 next
            next = setup_object(s, next);
            // 通过 p 的 freepointer 指针指向 next，
            //设置 p 的下一个空闲对象为 next
            set_freepointer(s, p, next);
            // 通过循环遍历，把 slub 中的空闲对象按照
            // 正常顺序串联在 freelist 中
            p = next;
        }
        // freelist 中的尾结点的 freepointer 设置为 null
        set_freepointer(s, p, NULL);
    }
    return slab;
}
```

内核在向伙伴系统申请 slab 之前，需要知道一个 slab 具体需要多少个物理内存页，而这些信息定义在 struct kmem_cache 结构中的 oo 属性中：

```
struct kmem_cache {
      ......
      // 其中低 16 位表示一个 slab 中所包含的对象总数，
      //  高 16 位表示一个 slab 所占有的内存页个数。
```

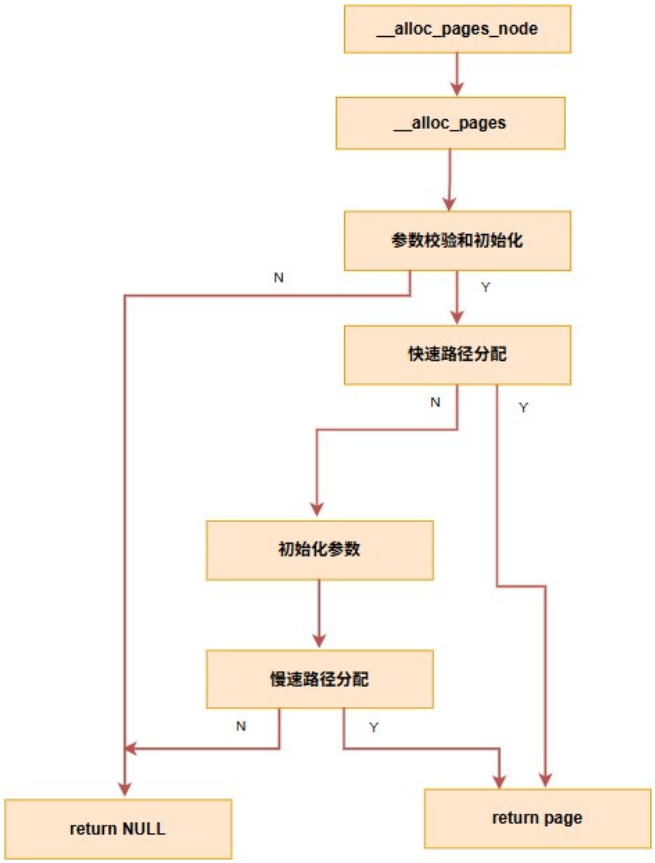
```
    struct kmem_cache_order_objects oo;
    ......
}
```

通过 oo 的高 16 位获取 slab 需要的物理内存页数，然后调用 alloc_pages 或者 __alloc_pages_node 向伙伴系统申请。




```
/*
 * Slab allocation and freeing
 */
static inline struct slab *alloc_slab_page(gfp_t flags, int node,
        struct kmem_cache_order_objects oo)
{
    struct folio *folio;
    struct slab *slab;
    unsigned int order = oo_order(oo);

    folio = (struct folio *)alloc_pages_node(node, flags, order);
    if (!folio)
        return NULL;
    slab = folio_slab(folio);
    __folio_set_slab(folio);
    /* Make the flag visible before any changes to folio->mapping */
    smp_wmb();
    if (folio_is_pfmemalloc(folio))
        slab_set_pfmemalloc(slab);
    return slab;
}
```

__alloc_pages 函数整体逻辑流程图：



详细分析过程，参考： Linux 物理内存分配全链路实现 章节。

如果当前 NUMA 节点中的空闲内存不足，或者由于内存碎片的原因导致伙伴系统无法满足 slab 所需要的内存页个数，导致分配失败。内核会降级采用 kmem_cache->min 指定的尺寸，向伙伴系统申请只容纳一个对象所需要的最小内存页个数。

```
struct kmem_cache {
    ......
    // 当按照 oo 的尺寸为 slab 申请内存时，如果内存紧张，
    // 会采用 min 的尺寸为 slab 申请内存，可以容纳一个对象即可。
    struct kmem_cache_order_objects min;
    ......
}
```

如果伙伴系统仍然无法满足，返回 NULL。接着通过 kasan_poison_slab 函数将 slab 中的内存用 0xFC 填充，用于 kasan 对于内存越界相关的检查。

```
#define KASAN_TAG_INVALID   0xFE /* inaccessible memory tag */
#define KASAN_SLAB_REDZONE  KASAN_TAG_INVALID
// 定义在文件：/mm/kasan/common.c
void __kasan_poison_slab(struct slab *slab)
{
    struct page *page = slab_page(slab);
    unsigned long i;
```

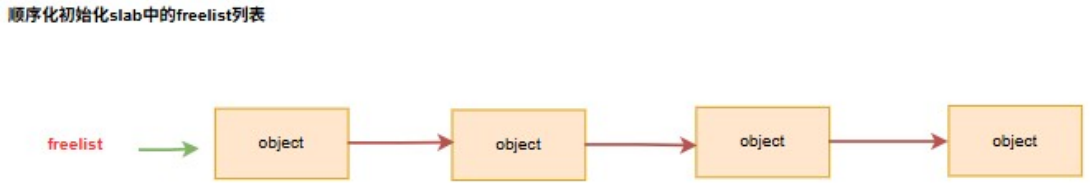
```
    // slub 可能包含多个内存页 page，挨个遍历这些 page
    // 清除这些 page->flag 中的内存越界检查标记
    // 表示当访问到这些内存页的时候临时禁止内存越界检查
    for (i = 0; i < compound_nr(page); i++)
        page_kasan_tag_reset(page + i);
    //用 0xFC 填充这些内存页的内存，用于内存访问越界检查
    kasan_poison(page_address(page), page_size(page),
            KASAN_SLAB_REDZONE, false);
}
```

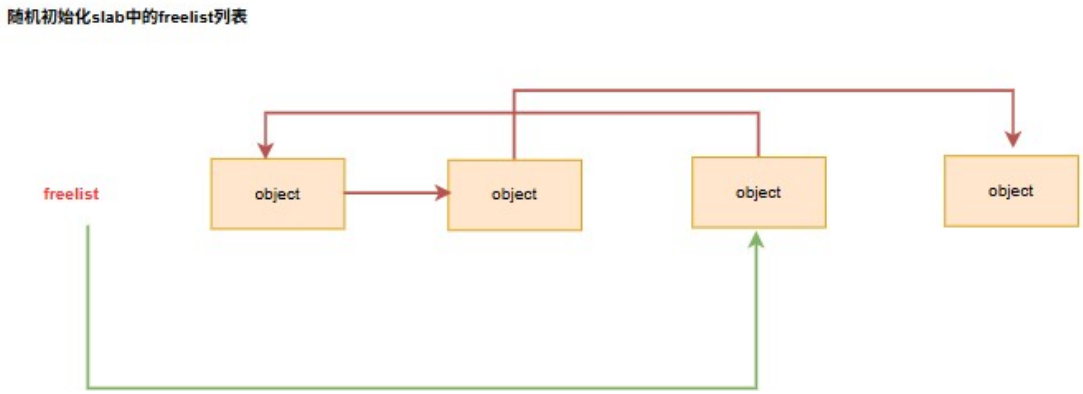
然后通过 shuffle_freelist 函数将内存页中空闲的内存块按照随机的顺序串联在 slab->freelist 中，并返回 true。如果随机化条件不满足，函数返回 false，然后会进入 if (!shuffle) 分支中，按照正常的顺序初始化 slab->freelist，将内存页中的空闲内存块通过 slab->freelist 链表组织起来。

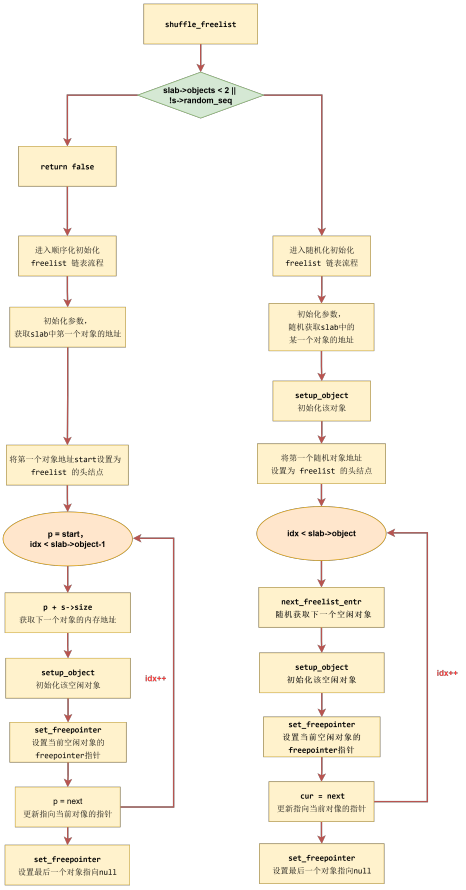
9.4 初始化 slab freelist 链表

内核在对 slab 中的 freelist 链表初始化有两种方式:

1. 一种是按照内存地址的顺序，一个一个的通过对象 freepointer 指针顺序串联所有空闲对象。
2. 另外一种则是通过随机的方式，随机获取空闲对象，然后通过对象的 freepointer 指针将 slab 中的空闲对象按照随机的顺序串联起来。







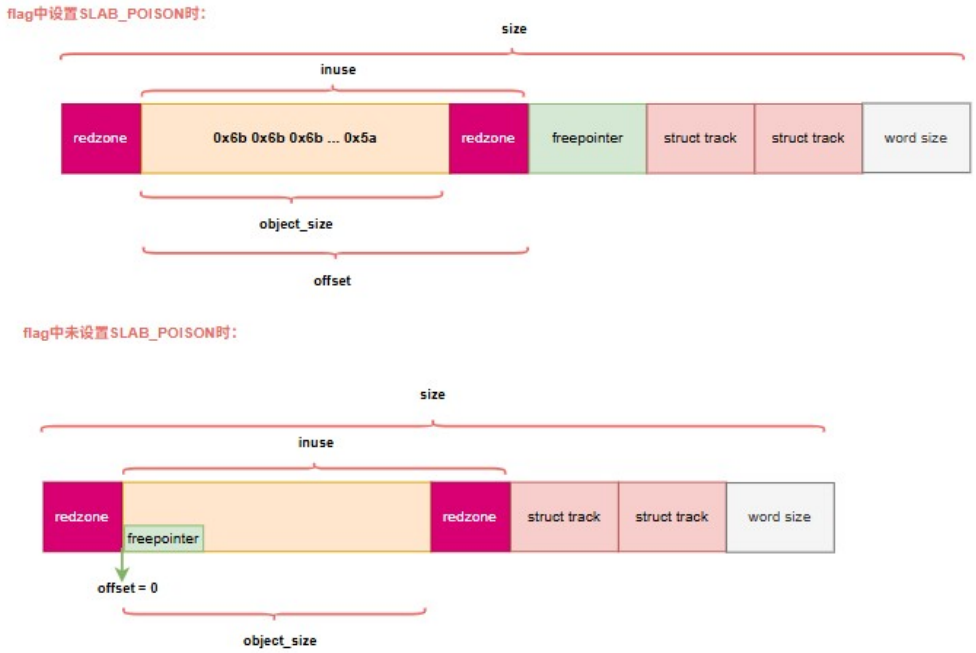
顺序初始化源码实现：

```
 // 获取内存页对应的虚拟内存地址
start = slab_address(slab);

shuffle = shuffle_freelist(s, slab);

// shuffle = false 则按照正常的顺序来初始化 freelist
if (!shuffle) {
// 获取 slub 第一个空闲对象的真正起始地址
// slub 可能配置了 SLAB_RED_ZONE，
//这样会在 slub 对象内存空间两侧填充 red zone，防止内存访问越界
// 这里需要跳过 red zone 获取真正存放对象的内存地址
    start = fixup_red_left(s, start);
     // 填充对象的内存区域以及初始化空闲对象
    start = setup_object(s, start);
 // 用 slub 中的第一个空闲对象作为 freelist 的头结点，
// 而不是随机的一个空闲对象
    slab->freelist = start;
  // 从 slub 中的第一个空闲对象开始，
  //按照正常的顺序通过对象的 freepointer 串联起 freelist
    for (idx = 0, p = start; idx < slab->objects - 1; idx++) {
```

```
            // 获取下一个对象的内存地址
            next = p + s->size;
            // 通过 p 的 freepointer 指针指向 next,
            //设置 p 的下一个空闲对象为 next
            next = setup_object(s, next);
            // 通过 p 的 freepointer 指针指向 next,
            //设置 p 的下一个空闲对象为 next
            set_freepointer(s, p, next);
            // 通过循环遍历，把 slub 中的空闲对象按照
            // 正常顺序串联在 freelist 中
            p = next;
        }
        // freelist 中的尾结点的 freepointer 设置为 null
        set_freepointer(s, p, NULL);
    }
```

内核在顺序初始化 slab 中的 freelist 之前，首先需要知道 slab 的起始内存地址 start，但是考虑到 slab 如果配置了 SLAB_RED_ZONE 的情况，那么在 slab 对象左右两侧，内核均会插入两段 red zone，为了防止内存访问越界。

在这种情况下，通过 slab_address 获取到的只是 slab 的起始内存地址，正是 slab 中第一个空闲对象的左侧 red zone 的起始位置。所以我们需要通过 fixup_red_left 方法来修正 start 位置，使其越过 slab 对象左侧的 red zone，指向对象内存真正的起始位置，如下图中所示。




```
void *fixup_red_left(struct kmem_cache *s, void *p)
{
    // 如果 slub 配置了 SLAB_RED_ZONE，则意味着需要再 slub
    // 对象内存空间两侧填充 red zone，防止内存访问越界
    // 这里需要跳过填充的 red zone 获取真正的空闲对象起始地址
    if (kmem_cache_debug_flags(s, SLAB_RED_ZONE))
```

```
        p += s->red_left_pad;
   // 如果没有配置 red zone，则直接返回对象的起始地址
    return p;
}
```

确定了对象的起始位置之后，对象所在的内存块也就确定了，随后调用 setup_object 函数来初始化内存块，这里会按照 slab 对象的内存布局进行填充相应的区域。初始化完对象的内存区域之后，slab 中的 freelist 指针就会指向这第一个已经被初始化好的空闲对象。

```
slab->freelist = start;
```

然后，通过 start + kmem_cache->size 顺序获取下一个空闲对象的起始地址，重复上述初始化对象过程。直到 slab 中的空闲对象全部串联在 freelist 中，freelist 中的最后一个空闲对象 freepointer 指向 null。

一般来说，都会使用顺序的初始化方式来初始化 freelist， 但出于安全因素的考虑，防止被攻击，会配置 CONFIG_SLAB_FREELIST_RANDOM 选项，这样就会使 slab 中的空闲对象以随机的方式串联在 freelist 中，无法预测。

随机初始化和顺序初始化唯一不同的点在于，获取空闲对象起始地址的方式不同：

1. 顺序初始化的方式是直接获取 slab 中第一个空闲对象的地址，然后通过 start + kmem_cache->size 按照顺序一个一个地获取后面对象地址。

2. 随机初始化的方式则是通过随机的方式获取 slab 中空闲对象，也就是说 freelist 中的头结点可能是 slab 中的第一个对象，也可能是第三个对象。后续也是通过这种随机的方式来获取下一个随机的空闲对象。

```
/* Shuffle the single linked freelist based on a random pre-computed sequence
*/
static bool shuffle_freelist(struct kmem_cache *s, struct slab *slab)
{
    // 指向第一个空闲对象
     void *start;
    void *cur;
    void *next;
    unsigned long idx, pos, page_limit, freelist_count;
    // 如果没有配置 CONFIG_SLAB_FREELIST_RANDOM 选项
    //或者 slub 容纳的对象个数小于 2
    // 则无需对 freelist 进行随机初始化
    if (slab->objects < 2 || !s->random_seq)
        return false;

    // 获取 slub 中可以容纳的对象个数
    freelist_count = oo_objects(s->oo);

    // 获取用于随机初始化 freelist 的随机位置
    pos = get_random_u32_below(freelist_count);
    page_limit = slab->objects * s->size;
    // 获取 slub 第一个空闲对象的真正起始地址
    // slub 可能配置了 SLAB_RED_ZONE，
```

```
    //这样会在 slub 中对象内存空间两侧填充 red zone，防止内存访问越界
    // 这里需要跳过 red zone 获取真正存放对象的内存地址
    start = fixup_red_left(s, slab_address(slab));
    /* First entry is used as the base of the freelist */
    // 根据随机位置 pos 获取第一个随机对象的距离 start 的偏移 idx
   // 返回第一个随机对象的内存地址 cur = start + idx
     cur = next_freelist_entry(s, slab, &pos, start, page_limit,
                freelist_count);
    // 填充对象的内存区域以及初始化空闲对象
    cur = setup_object(s, cur);
    // 第一个随机对象作为 freelist 的头结点
    slab->freelist = cur;
    for (idx = 1; idx < slab->objects; idx++) {
       // 随机获取下一个空闲对象
        next = next_freelist_entry(s, slab, &pos, start, page_limit,
            freelist_count);

        // 填充对象的内存区域以及初始化空闲对象
        next = setup_object(s, next);
        // 设置 cur 的下一个空闲对象为 next
        // next 对象的指针就是 freepointer，
        //存放于 cur 对象的 s->offset 偏移处
        set_freepointer(s, cur, next);
         // 通过循环遍历，就把 slub 中的空闲对象随机的串联在 freelist 中了
        cur = next;
    }
     // freelist 中的尾结点的 freepointer 设置为 null
    set_freepointer(s, cur, NULL);
     // 表示随机初始化 freelist
    return true;
}
```

9.5 初始化 slab 对象

内核按照 kmem_cache->size 指定的尺寸，将物理内存页中的内存划分成一个一个的小内存块，每一个小内存块即是 slab 对象占用的内存区域。setup_object 函数用于初始化这些内存区域，并对 slab 对象进行内存布局。

```
static void *setup_object(struct kmem_cache *s, void *object)
{
    // 初始化对象的内存区域，填充相关的字节，
    //比如填充 red zone，以及 poison 对象
     setup_object_debug(s, object);
    object = kasan_init_slab_obj(s, object);
```

```
    // 如果 kmem_cache 中设置了对象的构造函数 ctor，
    / 则用构造函数初始化对象
    if (unlikely(s->ctor)) {
        kasan_unpoison_new_object(s, object);
        // 使用用户指定的构造函数初始化对象
        s->ctor(object);
        // 在对象内存区域的开头用 0xFC 填充一段
        //KASAN_SHADOW_SCALE_SIZE 大小的区域
         // 用于对内存访问越界的检查
        kasan_poison_new_object(s, object);
    }
    return object;
}
```

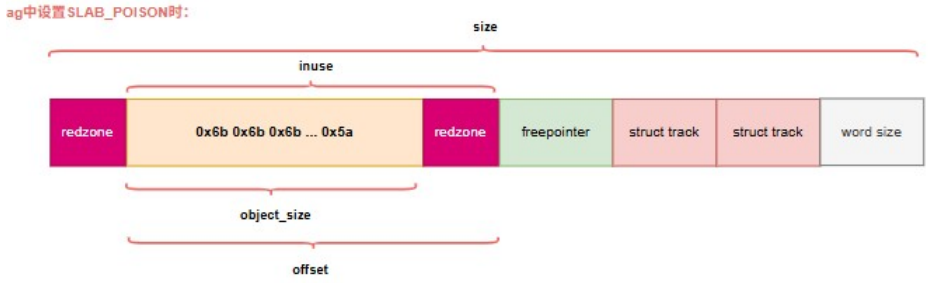
```
static __always_inline void kasan_poison_new_object(struct kmem_cache
*cache,
                           void *object)
{
    if (kasan_enabled())
        __kasan_poison_new_object(cache, object);
}
```

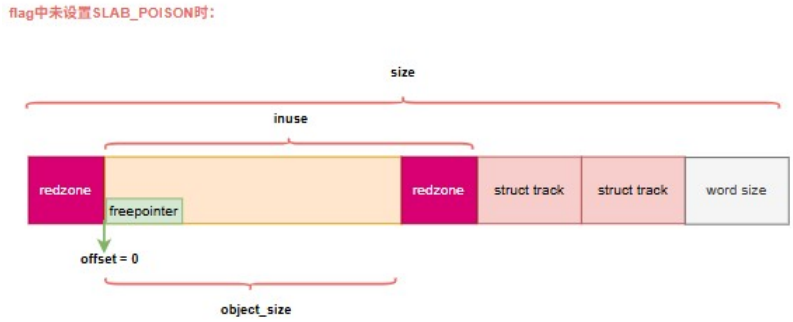
```
在文件：include\linux\kasan-tags.h 中
#define KASAN_TAG_INVALID   0xFE /* inaccessible memory tag */
在文件：mm\kasan\kasan.h 中
#define KASAN_SLAB_REDZONE  KASAN_TAG_INVALID
#define KASAN_GRANULE_SIZE  (1UL << KASAN_SHADOW_SCALE_SHIFT)

void __kasan_poison_new_object(struct kmem_cache *cache, void *object)
{
   // 在对象内存区域的开头用 0xFC 填充一段
      //KASAN_GRANULE_SIZE 大小的区域
    // 用于对内存访问越界的检查
    kasan_poison(object, round_up(cache->object_size,
KASAN_GRANULE_SIZE),
            KASAN_SLAB_REDZONE, false);
}
```

slab 对象内存布局的核心逻辑封装在 setup_object_debug 函数中：

```
在文件 include\linux\poison.h 中
#define SLUB_RED_INACTIVE   0xbb

/* Object debug checks for alloc/free paths */
static void setup_object_debug(struct kmem_cache *s, void *object)
```

```
{
   // SLAB_STORE_USER：存储最近访问该对象的 owner 信息，方便 bug 追踪
  // SLAB_RED_ZONE：在 slub 中对象内存区域的前后分别
  //填充一段 red zone 区域，防止内存访问越界
    // __OBJECT_POISON：在对象内存区域中填充一些特定的字符，
    //表示对象特定的状态。比如：未被分配状态
    if (!kmem_cache_debug_flags(s,
SLAB_STORE_USER|SLAB_RED_ZONE|__OBJECT_POISON))
        return;

    // 初始化对象内存，比如填充 red zone，以及 poison
    init_object(s, object, SLUB_RED_INACTIVE);

   // 设置 SLAB_STORE_USER 起作用，初始化访问对象的所有者相关信息
    init_tracking(s, object);
}
```

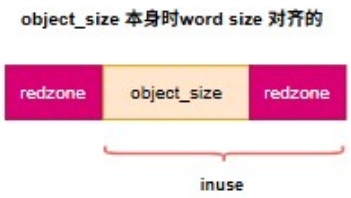




init_object 函数主要针对 slab 对象的内存区域进行布局，这里包括对 red zone 的填充，以及 POISON 对象的 object size 区域。

```
在文件 include\linux\poison.h 中：
#define POISON_END  0xa5    /* end-byte of poisoning */
#define POISON_FREE 0x6b    /* for use-after-free poisoning */
在文件 include\linux\slab.h 中：
#define SLAB_RED_ZONE       ((slab_flags_t __force)0x00000400U)


static void init_object(struct kmem_cache *s, void *object, u8 val)
{
    // p 为真正存储对象的内存区域起始地址(不包含填充的 red zone)
```

```
    u8 *p = kasan_reset_tag(object);
    //对象本身的内存占用的内存，不包含填充区
    unsigned int poison_size = s->object_size;
     // red zone 位于真正存储对象内存区域 object size 的左右两侧，
    // 分别有一段 red zone
    if (s->flags & SLAB_RED_ZONE) {
        // 首先使用 0xbb 填充对象左侧的 red zone
        // 左侧 red zone 区域为对象的起始地址到  s->red_left_pad 的长度
         memset(p - s->red_left_pad, val, s->red_left_pad);
        if (slub_debug_orig_size(s) && val == SLUB_RED_ACTIVE) {
            poison_size = get_orig_size(s, object);
        }
    }
    if (s->flags & __OBJECT_POISON) {
        // 将对象的内容用 0x6b 填充，表示该对象在 slub 中还未被使用
        memset(p, POISON_FREE, poison_size - 1);
         // 对象的最后一个字节用 0xa5 填充，表示 POISON 的末尾
        p[poison_size - 1] = POISON_END;
    }
    // 在对象内存区域 object size 的右侧继续用 0xbb 填充右侧 red zone
    // 右侧 red zone 的位置为：对象真实内存区域的末尾开始一个字长的区域
    // s->object_size 表示对象本身的内存占用，
    //s->inuse 表示对象在 slub 管理体系下的真实内存占用（包含填充字节数）
    // 通常会在对象内存区域末尾处填充一个字长大小的 red zone 区域
    // 对象右侧 red zone 区域后面紧跟着的就是 freepointer
    if (s->flags & SLAB_RED_ZONE)
        memset(p + poison_size, val, s->inuse - poison_size);
}
```

内核首先会用 0xbb 来填充对象左侧 red zone，长度为 kmem_cache-> red_left_pad。随后内核会用 0x6b 填充 object size 内存区域，并用 0xa5 填充该区域的最后一个字节。object size 内存区域正是真正存储对象的区域。

最后用 0xbb 来填充对象右侧 red zone，右侧 red zone 的起始地址为：p + s->object_size，长度为：s->inuse - s->object_size。如下图所示：